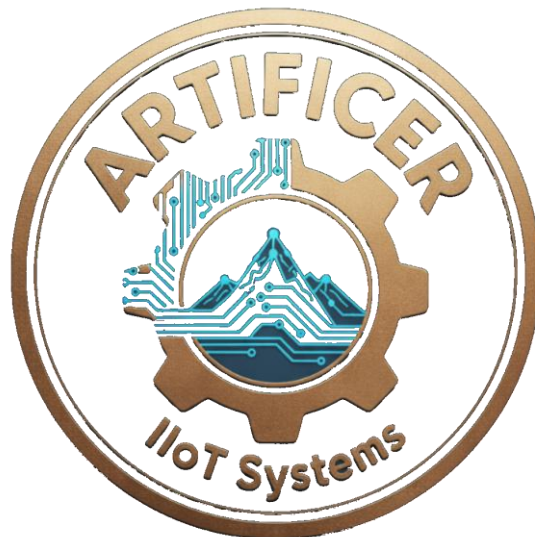


Salim Zakaria – Bachelor - Appendix



Content

My role in the project	4
Iterations	4
System Flow Diagram	4
First Iteration of the System Flow Diagram	4
Refined System Flow Diagram	7
Point system	9
Multiple Choice and True/False	10
Info box	11
Card Task	12
Click and Place Blazor	13
Purpose of the assignment	13
Why the Choices	16
The flow of the task	17
Click and Place – InfluxDB	18
Catch the Object	19
Storage	22
Lock system	23
System Engineering	23
System Structure	23
Flow system	25
Testing	25
Discussion	25
Microcontroller choice	25
Blazor vs JavaScript	26
MING	26
Source list	29

My role in the project

My role in the project was heavily related to the front-end using blazor, JavaScript, HTML and CSS. My work also related to having a huge part in the system engineering part. In this period did I also helped whenever I could, with taking critical conclusions, example choice of technology and microcontroller. I had the responsibility for most of the reports form the meeting with Knightec. I help structure the project by making a JIRA page, where a system where formed. I also had the responsibility for the task plan and the architecture of the system flow. Which where the backbone for the project.

Iterations

System Flow Diagram

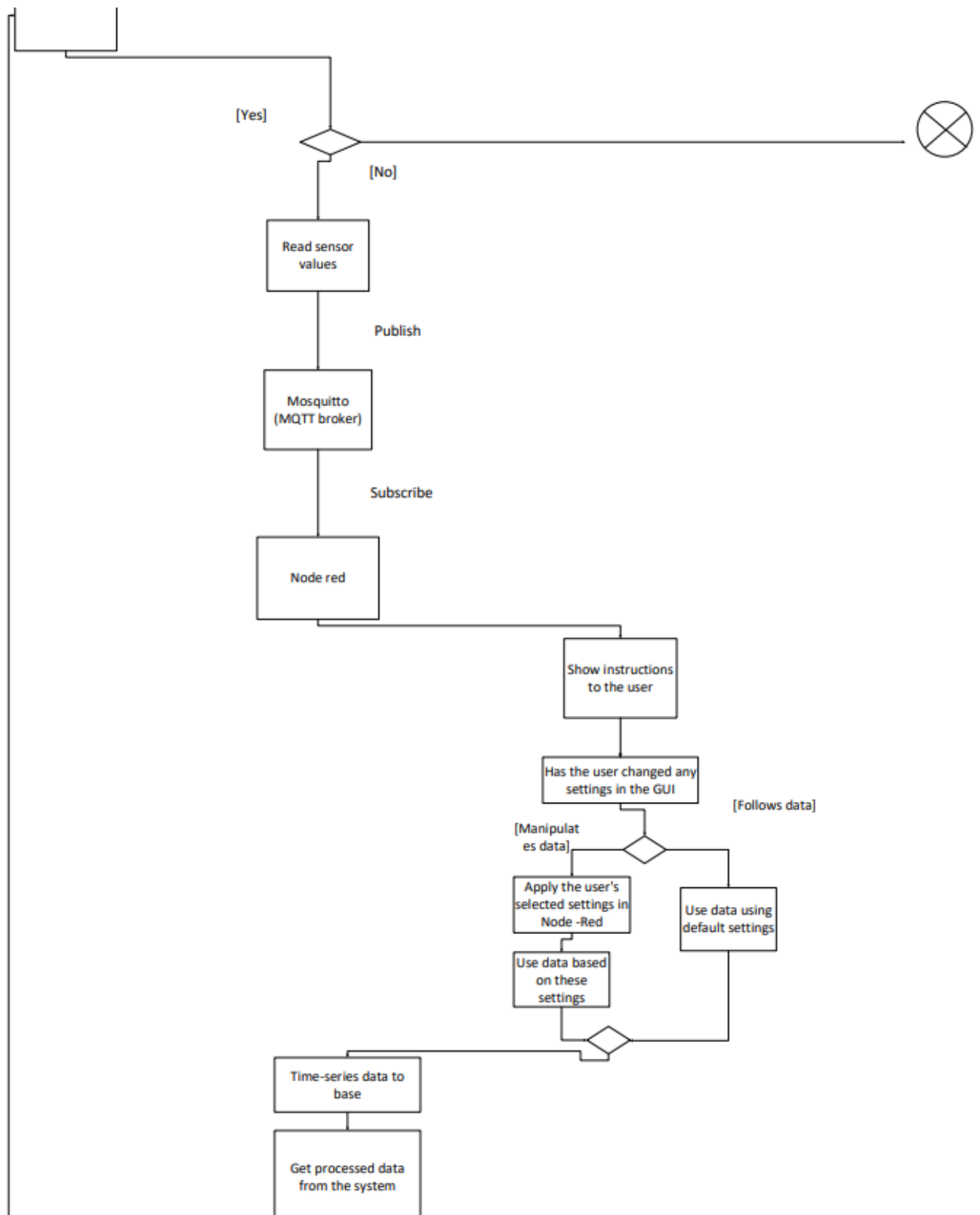
First Iteration of the System Flow Diagram

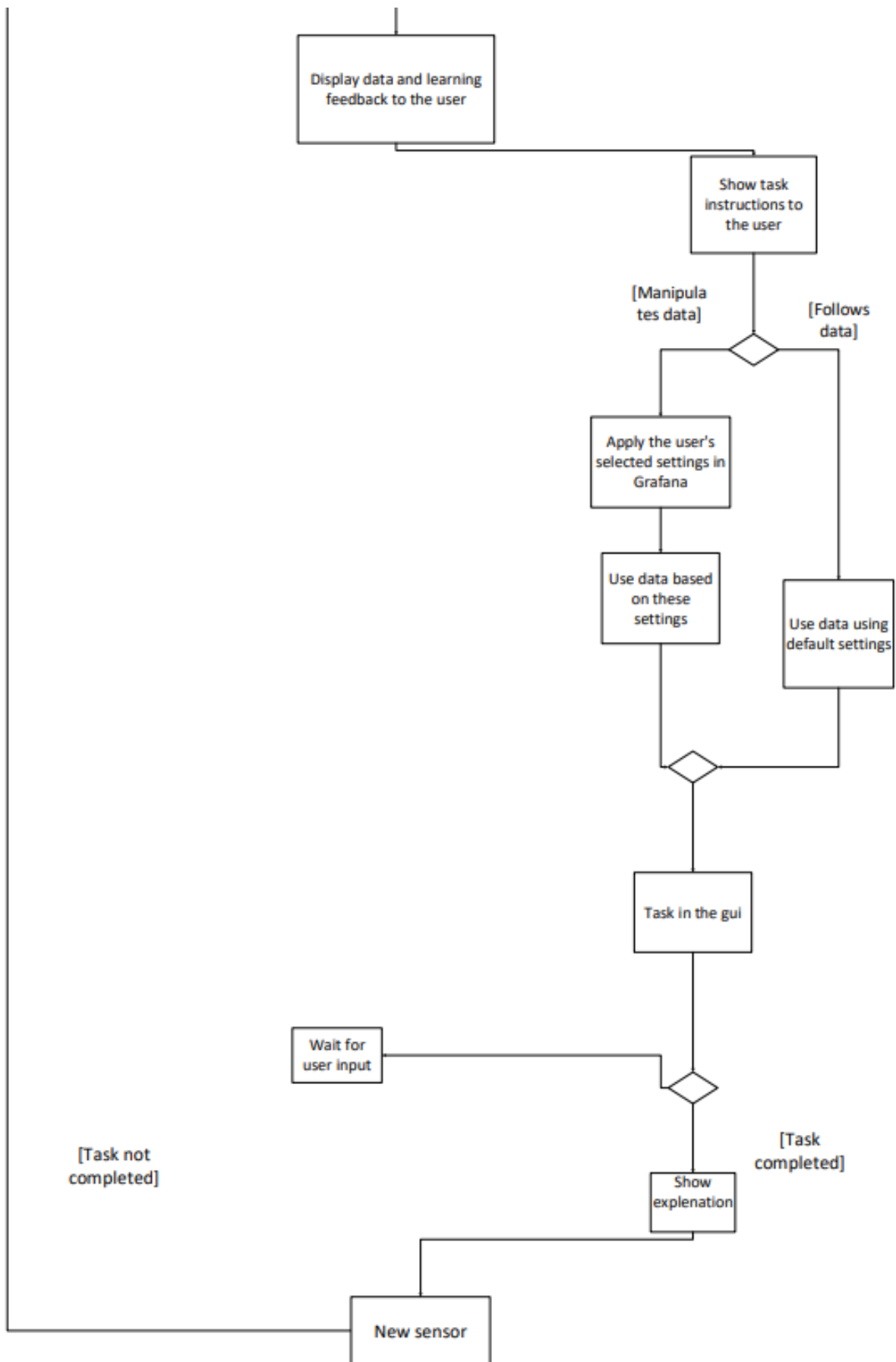
When we first receive the task, the group develops an activity diagram to illustrate the workflow and understanding of the system. It helps identify key components and visualize decisions within the system. The goal is to ensure that stakeholders are aligned. The diagram is the first iteration, developed in collaboration with stakeholders. The concept at that stage differs from what the project becomes later.

The box has the possibility to connect different physical sensors. It then takes raw sensor data and publishes it. The MQTT broker sends the data to relevant topics, and subscribers subscribe to those topics to receive the information. This process takes place in Node-RED. After that, the user has the option to either monitor the data or manipulate it, depending on the task.

The data is then saved in InfluxDB, and the user can either change settings in Grafana or simply observe the data, depending on the task. Tasks and explanations then follow, before the process starts again with a new sensor.

The key aspect that changes after several iterations with stakeholders is the concept itself. As the old diagram shows, the idea is to closely follow the MING stack, where sensors have an integral role. However, this approach is weak in terms of learning outcomes. Users do not learn enough from simply following the operation. By presenting the real MING stack directly, users are exposed to a large amount of code and technical details that are not relevant to their field, which can be demotivating and inefficient.



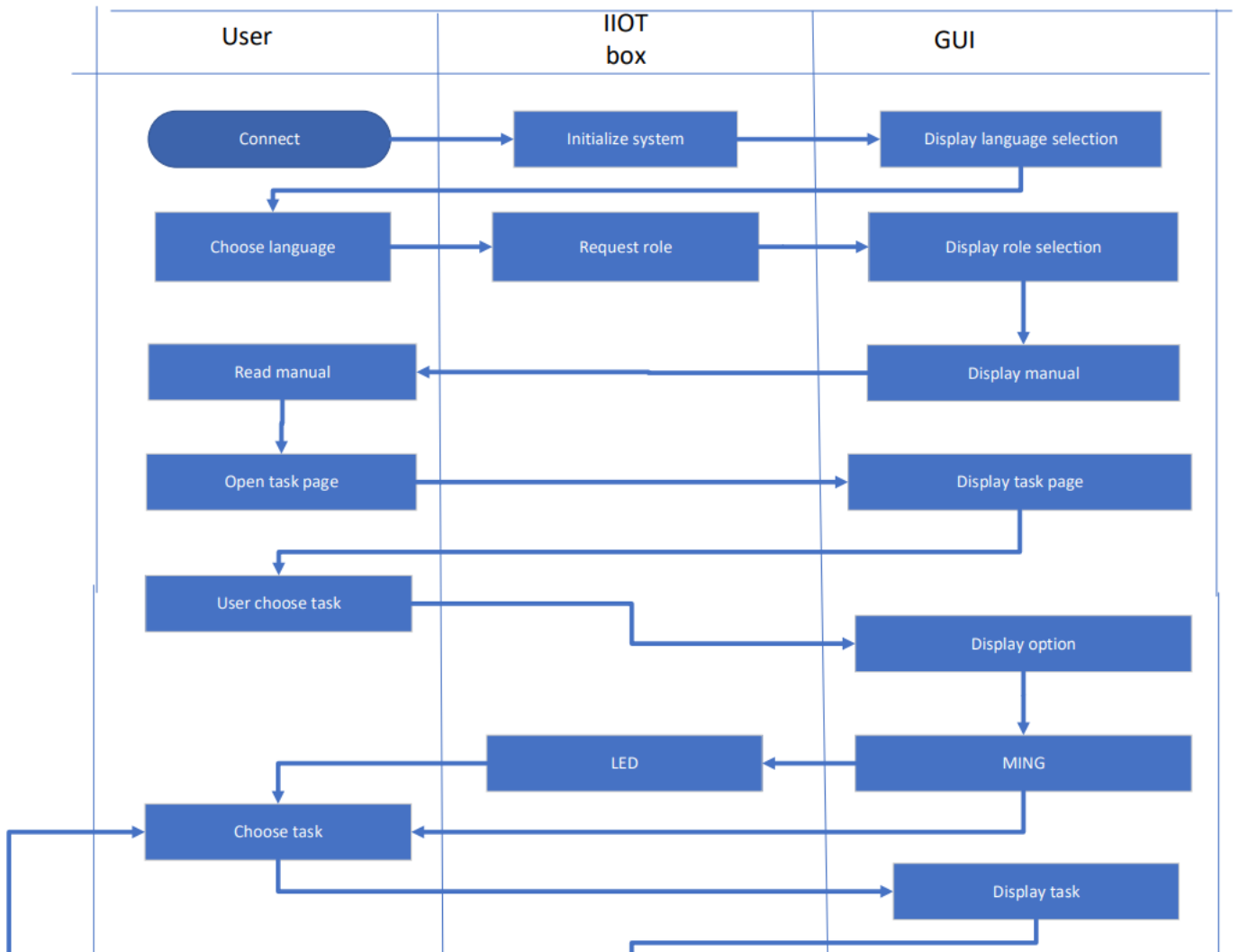


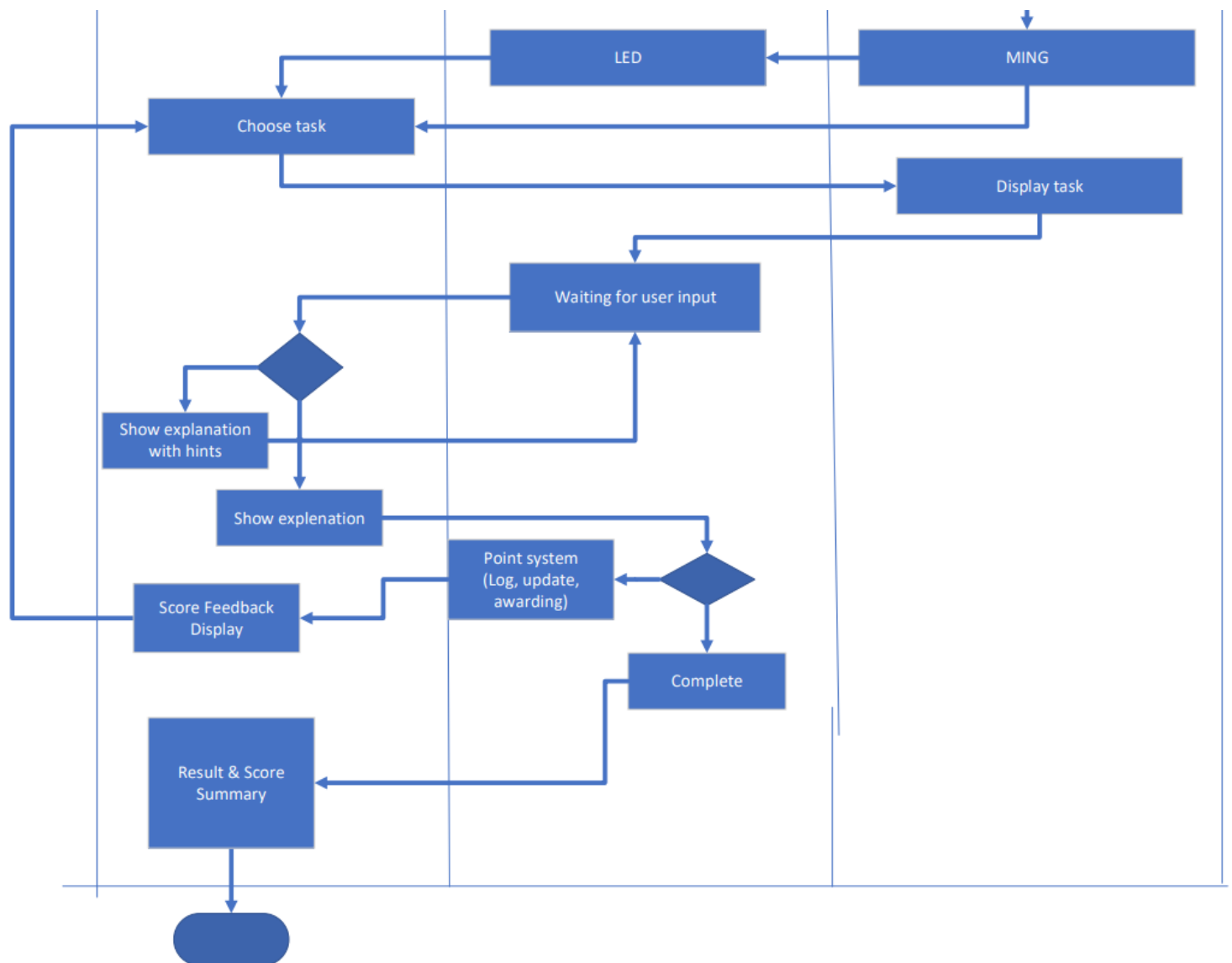
Refined System Flow Diagram

The refined flow diagram shows the new concept, where the sensors are less important, while the learning path and intuitive interaction become central. This is achieved by simulating the MING stack. By making the application itself the main part of the system, users are encouraged to learn in a more structured way. The new diagram includes improvements and optimized processes. It creates a better user flow, with the user, IIoT box, and GUI as the main components the system is built around. This makes the transitions between them clearer. The diagram continues to improve as new ideas are communicated with stakeholders or adjusted according to their needs. This creates a feedback loop that continuously improves the system.

The new diagram starts with the user connecting the box to begin the system. The user then has the option to select language through localization. These options are available in the top right side of the application. The GUI then makes a request for which manual to choose, either a student manual or a teacher manual. The teacher manual is more technical and contains additional information that is not relevant for students. The manual is displayed before the user selects a task. The available options are MQTT, Node-RED, Grafana, and InfluxDB. The physical LED light on the IIoT box lights up depending on where the user is located in the application.

A task description is then displayed, and the box waits for user input. An explanation is shown, and the user can also use hints. The point system is then initiated. With the point system, the user can keep track of which tasks have been completed. As tasks are completed, the point system updates and awards points. Score feedback is then displayed to the user. When all the tasks are completed, the user completes the application.





Point system

Previously, there was no point system. Points were not saved, so when users navigated between tasks, their scores were lost. The different tasks felt disconnected. A possible improvement would be to host a server where each user's total points are stored and displayed on a leaderboard. However, this is not implemented because it is outside the scope of the project. After testing when the tasks were connected to a joint score, the engagement of the user was much higher. The tasks point then gave a meaning. This is what point system solves.

Multiple Choice and True/False

The question tasks were designed differently. Like many modern quiz systems, only one question was displayed at a time to encourage focus.

HOME	OPPGAVETEKST	
MQTT		
NODE-RED	☐	Valg 1
INFLUX DB	☐	Valg 2
GRAFANA	☐	Valg 3
	☐	Valg 4
	☐	Valg 5
i		

Users could navigate forward and backward between questions. However, this approach could reduce engagement and become repetitive. In the current design, all questions are displayed on a single page. This allows users to view the entire task at once and receive immediate feedback upon submission. Correct answers are highlighted in green, helping users learn from the task while maintaining familiarity with the layout. In the earlier design, submitting the task would change the entire page layout, reducing this sense of familiarity.

HOME	OPPGAVETEKST
MQTT	
NODE-RED	
INFLUX DB	
GRAFANA	

Spørsmål 1?

Valg 1
Valg 2
Valg 3
Valg 4

Spørsmål 2?

Valg 1
Valg 2
Valg 3
Valg 4

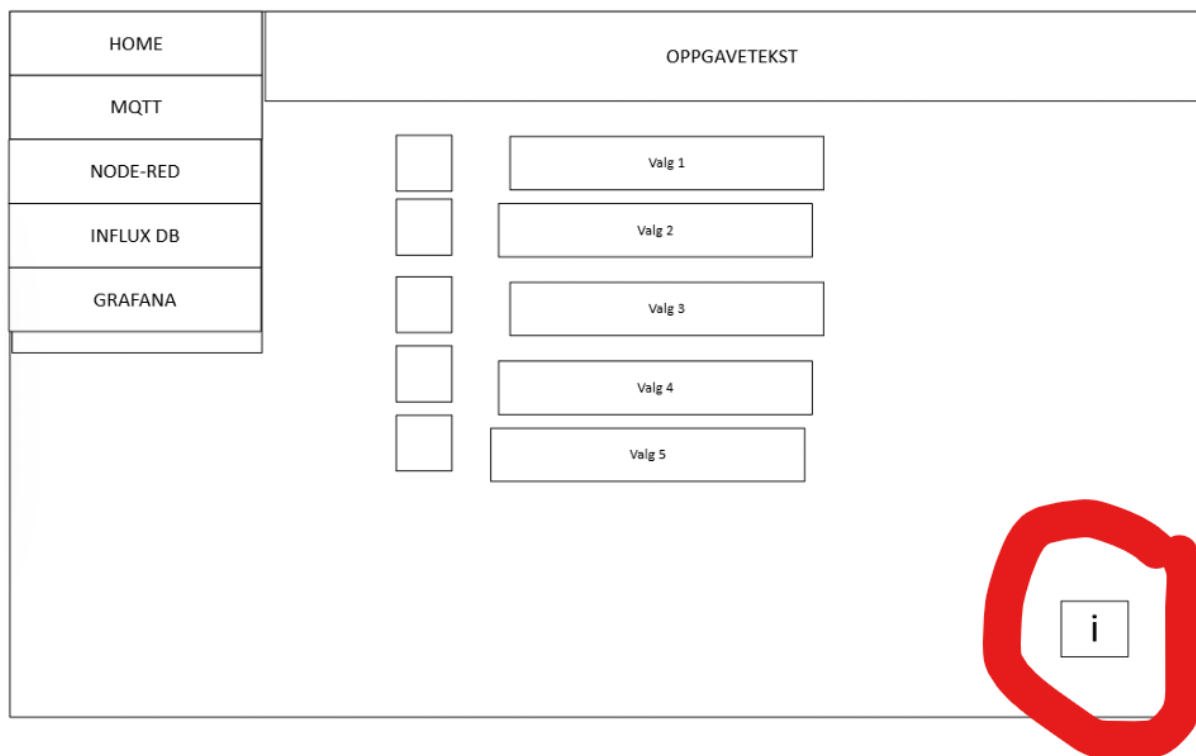
Spørsmål 3?

Valg 1
Valg 2
Valg 3
Valg 4

i

Info box

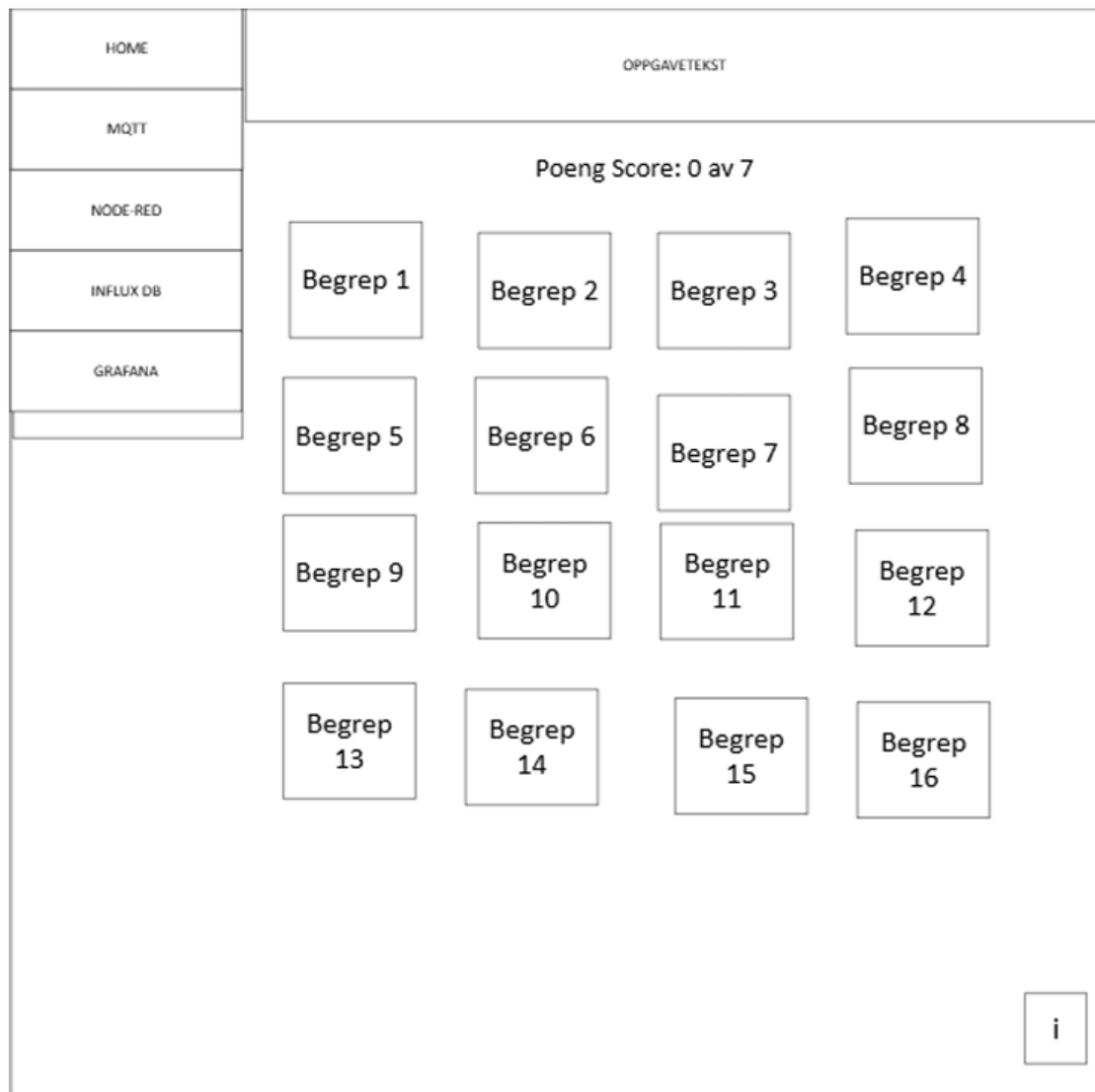
The info box idea emerges after testing, where users repeatedly ask: “What do I do here again?”. This happens because the user cannot return to the start page once the task is active.



The first design of the functionality is similar to the final product. The main decision is how the information should be displayed, and a popup is chosen as the best solution. Some challenges occur during development. Initially, the info box system uses multiple "else if" statements, but this is later improved with the current system. Before updating "infoBoxText" in the "go" function, the information from the previous task appears on the next task's start page, causing confusion. This is resolved by updating the information when the page changes. The size and color are adjusted to match the website, including rounded corners. A hover effect is considered, but using a pointer cursor is chosen as a better solution. The information provided by the info box is less useful for smaller tasks like multiple choice, as the content is similar. However, for larger and more complex tasks, the info box becomes more important. For example, in the Grafana "catch the object" game, the start page explains the purpose, scoring system, and how to complete the task. However, users may not read everything carefully and may forget valid ranges or rules. In these cases, the info box becomes useful. Overall, testing shows that the info box is most valuable for more complex tasks.

Card Task

The implementation of the task is inspired by digital flashcards. The task is originally designed so the user presses the terms that are relevant to MQTT, with a point score system. However, testing shows that this is not effective for learning, as users simply press different cards until the task is completed.



This is solved by adding an explanation on the back of each card so the user can learn. The response colors make the task more intuitive. The shuffle function is implemented so the user cannot see the position of the cards from others and copy. It also prevents users from memorizing positions, forcing them to learn the terms instead. The point score and the number of found cards are separated. This helps the user understand when the task is completed, which is also supported by the "completeBanner". Point deduction for wrong answers is included so users do not just click randomly to finish. The start page includes information so the user is able to solve the task. The flip animation is initially too fast, so it is adjusted. "classList" with "flipped" helps make the card static after it is selected.

Click and Place Blazor

Purpose of the assignment

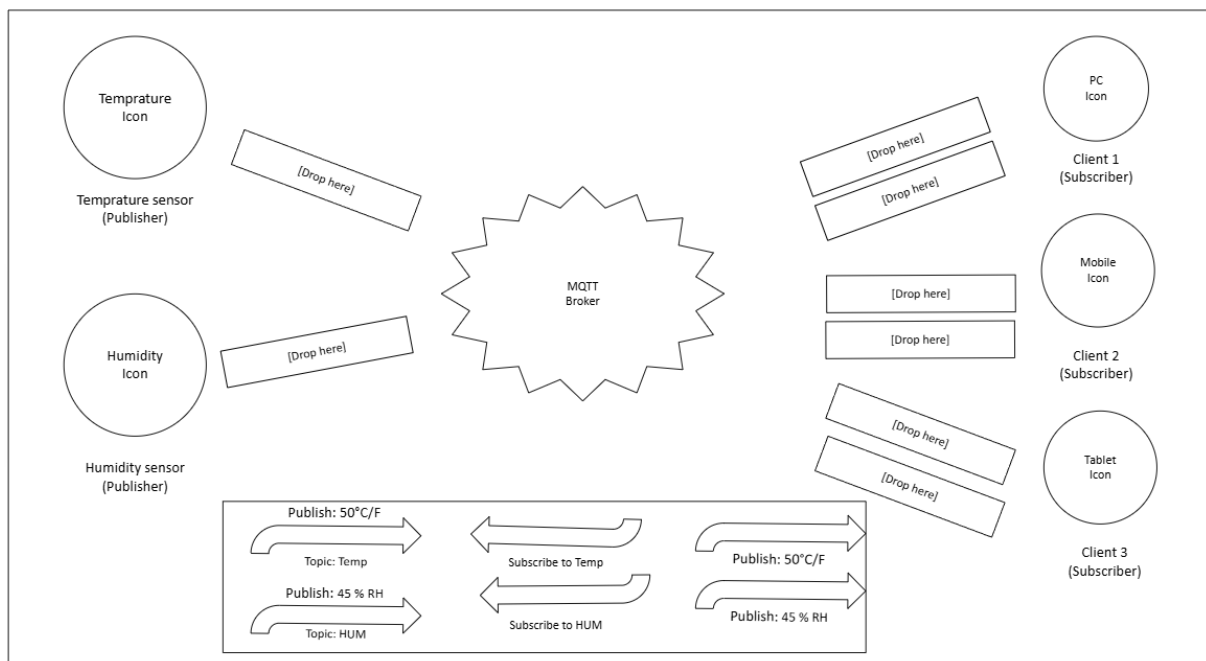
The first task I simulated was in c#. It was the first version of MQTT. The main goal of the MQTT select-and-place task is to give the user a better understanding of the MQTT flow.

It's designed to help the user learn the connection between the publisher, MQTT broker, and subscriber in an interactive way. We want the user to complete this task with a good understanding of the topic while also having fun learning.

The task starts with a description such as:

“Devices such as sensors first connect to an MQTT broker through a handshake. After the connection is established, the device can publish data to specific topics. Other clients can subscribe to topics to receive relevant messages. Temperature messages go to Temp subscribers. Humidity messages go to Hum subscribers.”

Before the start task button, there is an explanation of the theme of the task. It tells the user that a sensor connects to an MQTT broker through a handshake. Then the MQTT broker receives the published data from the sensors. The client can subscribe to the topic that is relevant to it and receive the message/data.



The task page is built up with the task title at the top. Under the title, do we find the task description box. Below that box are two bubbles on the left side, representing the publishers. There are two sensors: a humidity sensor and a temperature sensor. The bubbles are empty until a relevant icon is placed inside them. The drop zone underneath each bubble is where the arrow will be placed. It says “Send to Message broker”, which helps the user understand where to place the arrow.

In the middle of the page is the MQTT broker, with a select panel underneath it where the different arrow options are lined up. There are six options to choose from: “Publish: 50°C Topic: Temp” – Arrow pointed at the MQTT broker.

“Publish: 45\% RH Topic: Hum” – Arrow pointed at the MQTT broker

“Subscribe Topic: Temp” – Arrow pointed at the MQTT broker and client

“Subscribe Topic: Hum” – Arrow pointed at the MQTT broker and client

“Publish: 50°C” – Arrow pointed at the clients

“Publish: 45\% RH” – Arrow pointed at the clients

On the right side, there are three different clients. These are the subscribers that subscribe to the relevant topic, so it can receive the correct message/data. Each client has two drop zones. They are labeled with “Receive message” at the top and “Send to message broker” below it. The first drop zone expects the client to receive the correct message, while the under is where the topic the client subscribes to is sent to the broker.

The task description tells the user:

“Click an arrow from the Select panel, then click the correct drop zone to place it. Client 1 and Client 3 subscribe to Temperature. Client 2 subscribes to Humidity only.”

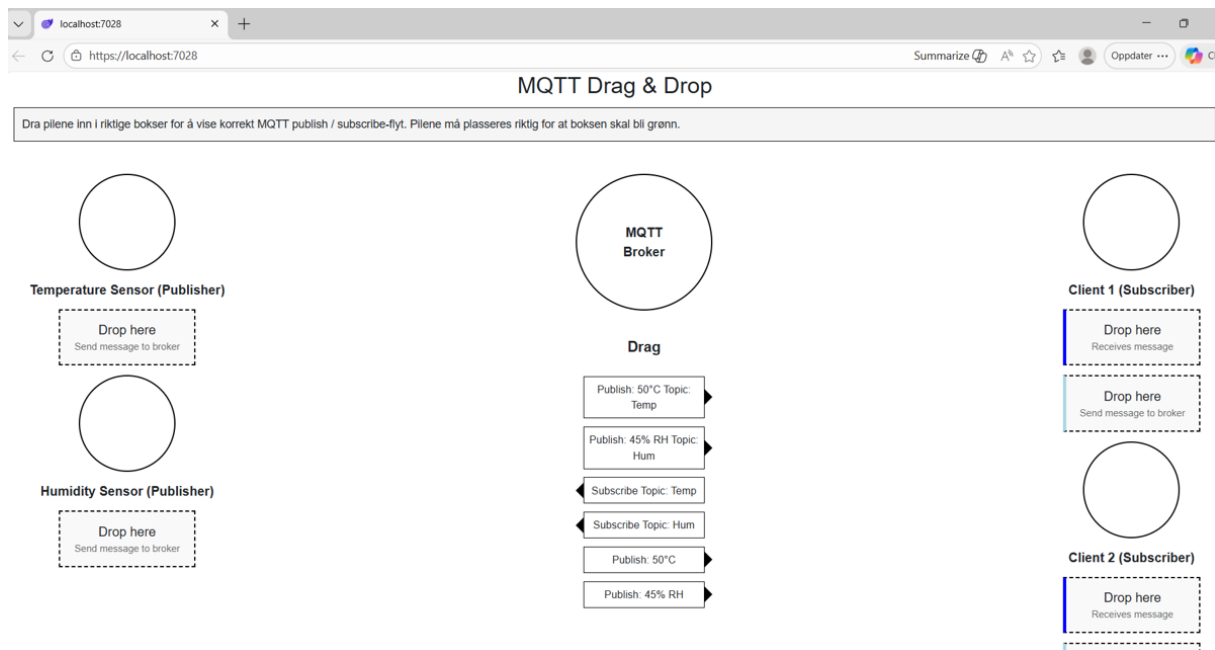
As seen in the top section of the task page, the aim of the task is to click and place the correct arrow boxes from the select panel below the MQTT broker into their correct drop zones. Client 1 and 3 need temperature data, while the other client needs humidity data. If a drop zone receives the wrong arrow, it turns red and the arrow returns to the select panel. If it is correct, it turns green and locks in place until the task is complete. The user can take as many arrows from the same option as needed.

The targeted users of this task are vocational school students who want to learn about the MQTT flow. The users should already have some basic understanding of what a broker does, as well as the roles of subscriber and publisher. That is why easier tasks are introduced beforehand to establish a basic understanding before they begin with this task. There is a balance between technical difficulty and usability. The task needed to be presented in a way that would keep the user engaged while also teaching them something. It is easy to get started and understand how the task works, but the user still needs to know the different terms so that it can complete it successfully.

The assignment contributes to learning by showing the complete flow between the MQTT broker, publisher, and subscriber. Their roles are central to MQTT communication. Through this task, users gain a better understanding of what the clients need to subscribe to and what the publisher sends to the broker. The user will also develop an understanding of how the topic filters and sends messages between the publisher and subscribers.

This task is relevant for real-world applications, as well as industry relevance for vocational school education. MQTT is an integral part of the Industrial Internet of Things. The MQTT flow can be used to show how messages are forwarded in everyday applications. It is also used in production and analysis systems by ingesting data information and processing it for relevant systems.

Why the Choices



When it was decided to simulate the MING stack, we agreed to start with MQTT. The MQTT flow is an important part, and it could be represented in several ways. It started with a sketch of a drag-and-drop task. The customers had a positive reaction to it, as they wanted a drag-and-drop task because it would be interactive for the user.

I first programmed it in Blazor WebApp using C#, and it turned out well design-wise. When implementing the application, it became more user-friendly to line up the arrows under the MQTT broker and place the drop zones under the circles. This made the application seem/appear more professional. However, drag-and-drop functionality was not ideal in Blazor. It had issues with the drop zones, as the arrows would not always drop correctly. The drop zones also failed to change color consistently. The page became long, and dragging an arrow while scrolling up and down was difficult. This created precision problems, especially on a smaller screens.

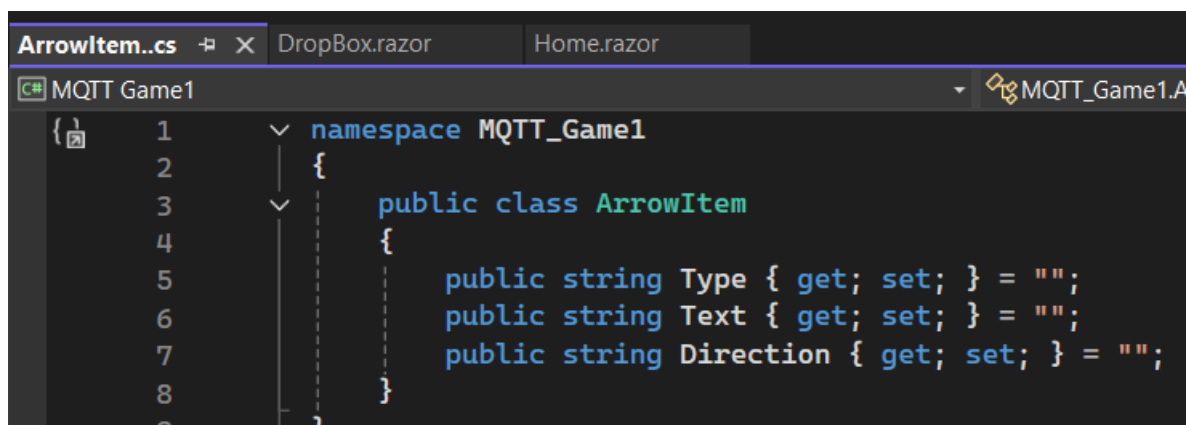
The main issue at the time was storage on the microcontroller. The Blazor page required a lot of storage, which caused lagging, bugs, and unpredictable behavior. When it was decided to code in VS Code and Java, the focus shifted much more toward the front end. After testing, the conclusion was that drag-and-drop should be changed into a click-and-place application. This reduced storage usage significantly. The new application became much simpler and more consistent. In the future, it could be modified back if there is enough time or adapted for another task.

Through extensive testing and iterations with mechanical students and stakeholders, user testing provided valuable feedback through observation. The earlier tasks that provide a basic understanding are more informative and repetitive, such as multiple-choice tasks. Compared to passive learning methods, it was seen as positive that later

tasks had a different type of structure. This increased both engagement and motivation for the users. Testing with students and customer dialogue led to many changes. The background color was changed from light to dark, which made the circles and colors stand out more. This also led to changes in font and text color.

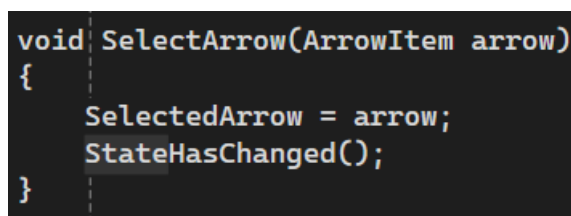
There was a request to include some explanation of the theme in the task box. This was solved by adding a start page that explains the topic before entering the task. Client 1 and 3 subscribe to the same topic in this task, so having the same arrow twice was misleading. This was solved by allowing unlimited arrow options while only locking in the correct ones. It is more interactive to let the drop zones turn red and green, making the application more intuitive. The colors also help make the user more engaged. There was also a request for a clear indication when the task is completed. A confirmation can be seen in. Another piece of feedback received was that “subscribe to topic” should be shown in both directions, because it represents a two-way handshake between the client and the broker and makes the direction of data flow more visual.

The flow of the task



```
namespace MQTT_Game1
{
    public class ArrowItem
    {
        public string Type { get; set; } = "";
        public string Text { get; set; } = "";
        public string Direction { get; set; } = "";
    }
}
```

The blazor program is split into 3 files. ArrowItem.cs creates the content of the arrow boxes, such as direction, text and type. Home.razor is where the styling happens, it checks the response and handles which arrow that are selected. DropBox.razor is where the drop zone logic happens.



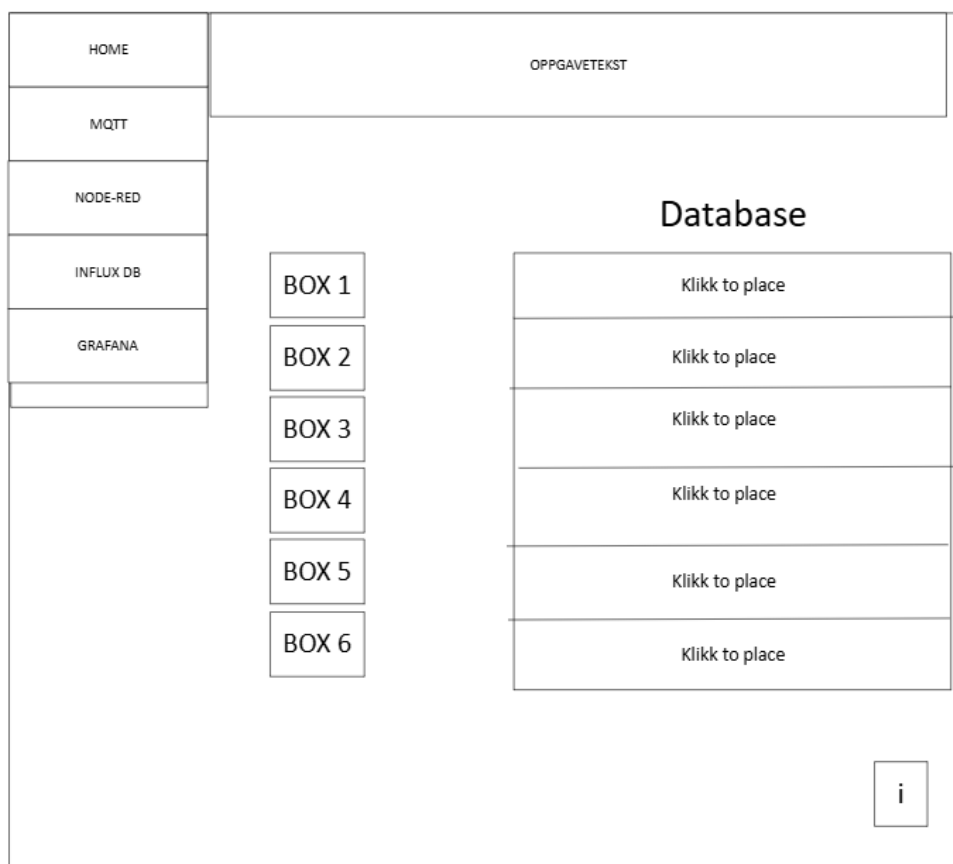
```
void SelectArrow(ArrowItem arrow)
{
    SelectedArrow = arrow;
    StateHasChanged();
}
```

It starts with Home.razor creating the startpage, then it defines the arrow boxes with ArrowItem. Then the game starts and the task opens. When the user selects a option, the selectArroe saves the chosen arrow in selectedArrow. Then the StateHasChanged happens, which updates the page since Blazor handles the logic automatically instead of manually like JavaScript. This means JavaScript has to constantly check all the 8

dorpzones with different data.expect values. StateHasChanged sends the selected arrow to all the dropboxes at once. Then when a drop box is clicked, PlaceArrow places it and checks if the type matches the expected value. If correct, the zone gets locked, and it becomes green. The result is sent Home with InvokeAsync. Then Home receives the answer with await in HandleResult. It updates the message and counts points up or down. The feedback color is hidden after 1 second. After all 8 are correct the complete banner is shown. It uses the same logic as the recent one.

Click and Place – InfluxDB

The task page is placed as right and left side. The box should consist of sensor data, where the right side should be database. The task should be as click and place.



The challenge I had was how an InfluxDB database could be intuitive and realistic at the same time. I wanted to reuse the click and place logic that I was familiar with and that the user could also use. By testing, the users didn't like new formats. So I needed to change the score system and deduct points because this is easier and they already know the format. I came to the conclusion that the sensor values are timestamped and need to be placed in the right order. To make it fun, the values are hidden, then revealed after 5 seconds after each reveal. We had a temperature sensor on the box without a sensor. This was the perfect opportunity to make a task with a real sensor, rather than

only simulated. It was a bit difficult to understand how to connect with sensor code that was made by someone else. This was the biggest challenge.

Catch the Object

In the older iteration the task was different, starting with the CSS.

```
#spill {
  width: 400px;
  height: 500px;
  margin: auto;
  background: var(--bg2);
  position: relative;
  overflow: hidden;
  display: none;
}
```

```
#page-grafana-t3 canvas {
  width: 900px;
  height: 450px;
  margin: auto;
  display: block;
}
```

These two lines made the game frame and the result canvas size static. When this was tested on bigger screens, the user experience was not the same. The screen looked small.

```
let temp=0,fuktighet=0,Vibrasjon=0,feil=0;
```

```
const typer=["temp","Fuktighet","Vibrasjon"];
const type=typer[Math.floor(Math.random()*typer.length)];
```

The other code did not use a graph, only bar and pie charts. The values that were counted were temperature, humidity, vibration and errors. In this task it took in 4 different values. It used a random value between the 3 sensors.

```
// temperatur
if(type==="temp"){
  if(Math.random()<0.1){ // 10% sjanse for galt
    verdi="ERROR";
    gyldig=false;
  }
  else{
    let x=Math.floor(Math.random()*50);
    verdi=x+"°C";
    if(x<9||x>27)gyldig=false; // kun 9-27 er gyldig
  }
}
```

This is how the temperature sensor worked: there was a 10% chance that an error would occur and this was invalid. Then a random temperature between 0 to 49 would come out. Only the values between 9–27 are valid. This is because it is the standard temperature range in the industry.

```
// Fuktighet
else if(type==="Fuktighet"){
  if(Math.random()<0.15){
    verdi="NaN";
    gyldig=false;
  }
  else{
    let y=Math.floor(Math.random()*61);
    verdi=y+"%RH";
    if(y<40||y>60)gyldig=false; // kun 40-60 er gyldig
  }
}
```

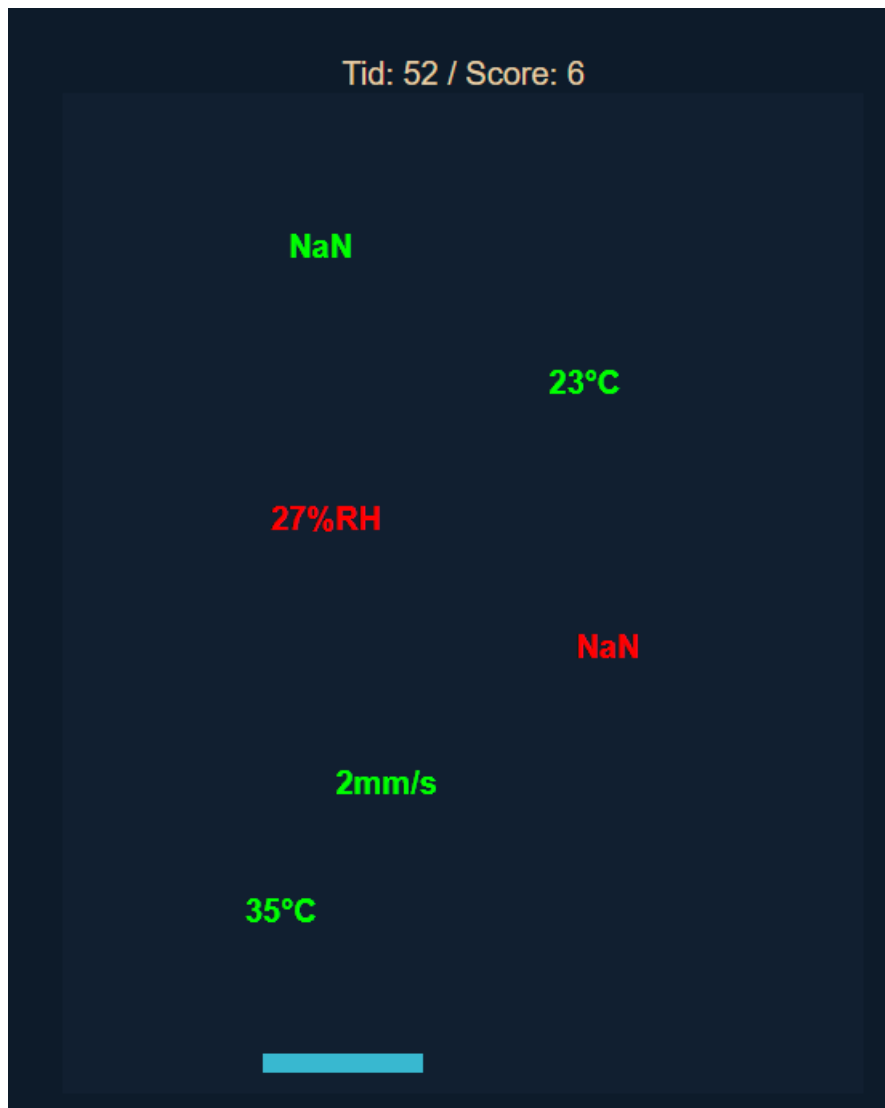
The same logic was implemented in the humidity data. There was a 15% chance that NaN would come out, which is invalid. Values between 0 to 60 would come out, where those under 40 are invalid. This is a good zone in the industry.

```
// vibrasjon
else if(type==="Vibrasjon"){
  if(Math.random()<0.15){
    verdi="ERROR";
    gyldig=false;
  }
  else{
    let z=Math.floor(Math.random()*11); //går fra 0-10
    verdi=z+"mm/s";
    if(z>5)gyldig=false; //kun 0-5 er gyldig
  }
}
```

The same applied to the vibration data. There is a 15% chance of an error. Values between 0–10 would come out, where values over 5 are invalid. 0 to 5 mm/s is a good zone for real machines in the industry.

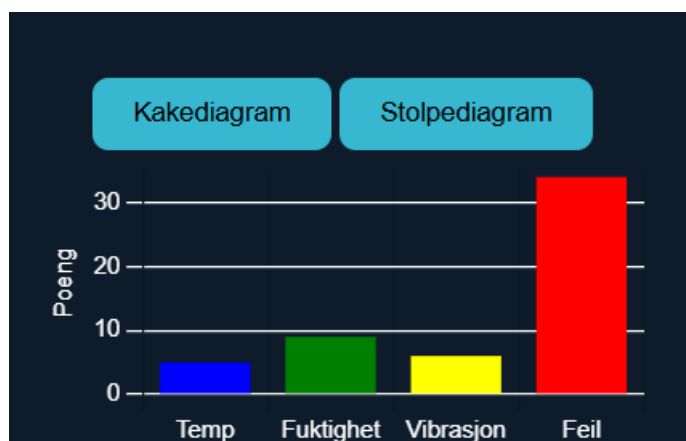
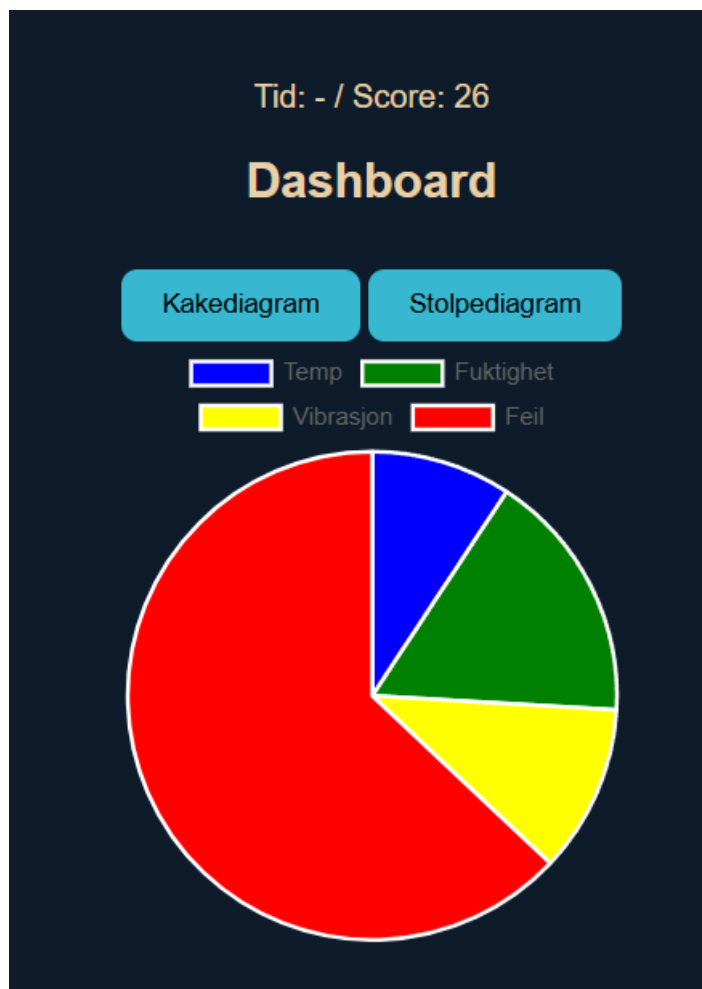
```
//Farger på verdier
if(gyldig){
  el.style.color="lime"; // gyldig er grønn
}
else if(Math.random()<0.25){
  el.style.color="lime"; // 25% av feil sensorverdi blir grønne
}
else{
  el.style.color="red"; // resten blir røde
}
```

If the sensor values were valid, the data was shown in green. 25% of the invalid values were marked as green. This was to add noise, which happens in the industry. Some invalid values come out as valid. The rest of the invalid data are represented as red. As seen in the picture, "NaN" is represented as green (valid).



There were a few things that led me to change the structure. When testing on 4 people around the age of 30, where 2 of them were machine engineers, our targeted group. Their response was the same from all four:

- There was too many things to think about in the short amount of time.
- They went quickly down to negative points, which affected their motivation, that's why I had +3 points.
- It was a bit boring with only the bar and the pie chart, a graph was missing.
- 3 values was difficult, that's why I chose to focus on the humidity. This is where most of them got the points.
- They didn't want me to use the inconsistent values as 9 to 27 rather 10 to 30. Still then it was difficult.
- Too much colors
- I felt that timestamp graph was missing, which gives the Grafana feel.
- The object felt too fast.



Storage

It was discussed how the data would be saved. Cookies were considered, but the system does not use a server. Cookies save data by sending it to a server. This would have been useful if the system handled sensitive data, as cookies could also secure it by adding a login feature. Local storage reads and writes data, so this was perfect for our system. Session storage has the same functions as local storage, but the data is removed when the user closes the browser. I was afraid the storage would take too long,

but local storage solved our problem perfectly in this system. I also thought resetting the storage would be complex, but it was not.

Lock system

The lock system was an exciting topic — this is where the learning path was created. It came late in the project, though it was something we had been discussing early on. When I first implemented it, I only locked the menu buttons, but if the user pressed the MQTT button, the task would be available another way. This is why I used the alert function, for extra security. The unfinished tasks in the menu were also shown in red at one point. Through testing, I was told there were too many colors, which I agreed with. They were then changed to white.

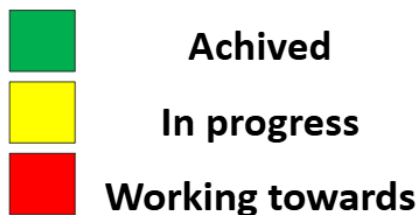
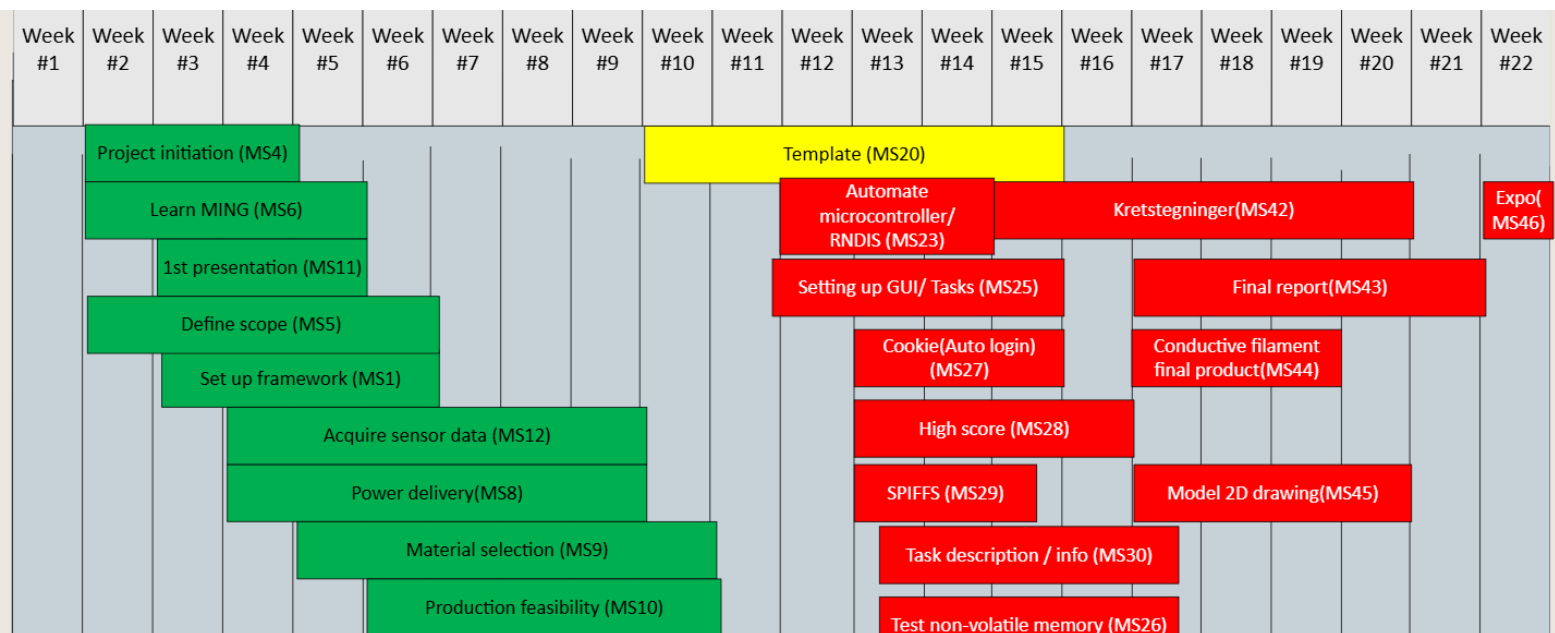
System Engineering

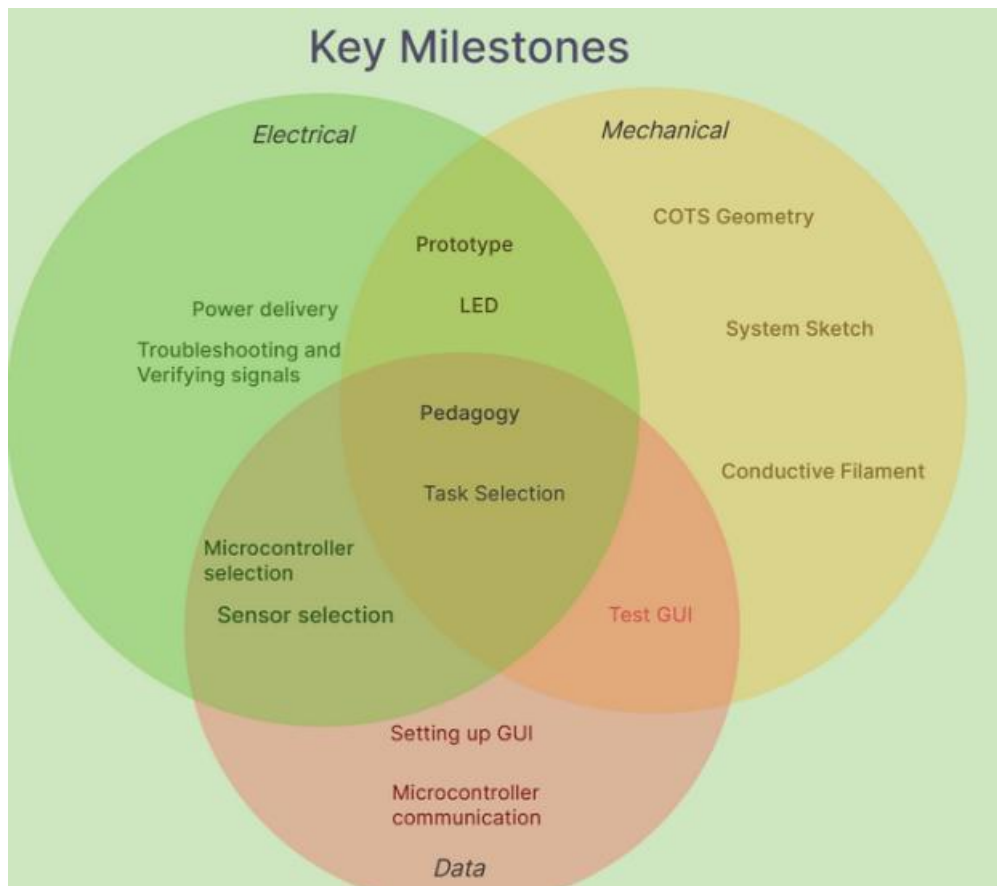
System Structure

In the beginning of the concept was I heavily involved in the system engineering. I was actively working with the stakeholder requirements and system requirements. By this I made something called milestones also known as task. Which derivates form the system and stakeholder requirements. That made us a clear plan with task we follow in the second part of the project. The first part with Jira did not go well because of group dynamic and people wanted to work in their own way. So the JIRA was something we moved away from, to the task plan which was a more freely way to work. These tasks/goals where written on the white board and crossed when it was complete. This made the team back in track again. I also made a easy figure early to show how the discipline would work together.

ID	Milestones	Derived from	JIRA Task	Achieved (Week
MS1	Set up framework	Vs8, Kt37	SCRUM-90, SCRUM-77	Week 6
MS2	Select microcontroller	Vs8, Kt7, Kt1	SCRUM-74	Week 10
MS3	Select sensors	Vs19, Kt39, Kt42	SCRUM-94, SCRUM-96	Week 11
MS4	Project initiation	US27, US28, US31	SCRUM-17, SCRUM-21	Week 4
MS5	Define scope	US31, Kt9, Vs18	SCRUM-14, SCRUM-29	Week 6
MS6	Learn MING	Vs8, Kt9, Vs17	SCRUM-45, SCRUM-68	Week 5
MS7	Sketch of the system	US28, Kt11, Vs13	SCRUM-64, SCRUM-65	Week 10
MS8	Power delivery	Kt38, Kt1, Kt3	SCRUM-73, SCRUM-74	Week 7
MS9	Material selection	Vs22, Vs23, Kt6	SCRUM-97	Week 10
MS10	Production feasibility	Kt6, Kt11, Vs12	SCRUM-97, SCRUM-96	Week 14
MS11	1st presentation	US31, kt10, Kt9	SCRUM-54, SCRUM-55	Week 5
MS12	Acquire sensor data	Kt39, Kt40, Vs19	SCRUM-68	Week 9
MS13	Localization	uv15, uv33	SCRUM-85, SCRUM-90	Week 11
MS14	2nd presentation	US31, kt10, Kt9	SCRUM-89	Week 13
MS15	Troubleshoot and verify signals	Kt39, Kt40, uv36	SCRUM-94, SCRUM-100	Week 14
MS16	1st prototype	Kt2, Kt3, Vs13	SCRUM-96, SCRUM-89	Week 10
MS17	Procurement	Kt6, Kt11, Vs23	SCRUM-80, SCRUM-101	Week 11
MS18	COTS geometri	Vs13, Kt11, Vs23	SCRUM-92	Week 15
MS19	Conductive filament validation	Kt11, Vs19, Kt42	SCRUM-87	Week 15

These are the tasks which were a result from the requirements.





This how the disciplines would work with the tasks together.

Flow system

I also made the flow diagrams and had responsibility for them. The flow and the function can be read in the main report.

Testing

I tested on many machine engineers and people older than me who had no relation about the MING stack. They loved the result and the feedback was positive. The aim was to make them learn about MING by having fun. By using the test it influenced by system better. This can be at Iterations or “testing” chapter in the main report. This improved the system and the user experience.

Discussion

Microcontroller choice

When we learned and had an understanding of what the MING consists of, then we did research about which microcontrollers would not only be the best. It must fulfill the requirements of the customers, which is to find a cheap microcontroller. We did research, and we went for the esp32. It was cheap, accessible and supports wifi. So we

would have a backup, if the communication without wifi would not work. Then we chose to program in Blazor. The esp32 could not host the website. The esp sends the data and Blazor gets the data and shows it in the web. Then we found the new version which was esp32-s3. This version is still cheap and has more storage, which was a problem that we feared. It has the usb connection to the pc, this will make data communication. Native usb is what direct data communication with a pc with usb. What was the most important for me was the storage and what I had access to.

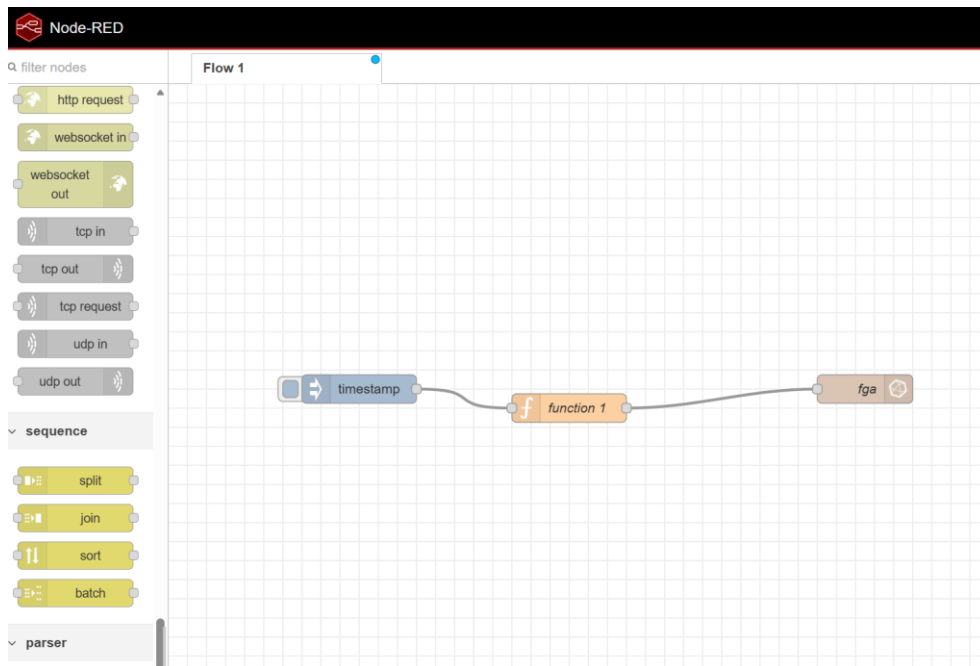
Blazor vs JavaScript

When we found out that the project was to simulate the MING. We came across a huge decision. It was either to write the program in Blazor or JavaScript. I had a preference with C#, because I used it in the internship and I liked the balance between frontend and backend. It is easier to implement as the code logic is structured in files. When I learned more about HTML and JavaScript the more I liked it. I learned a new way to code. Where the design is integral. I liked that every detail had such an impact. While it is complex, it is good to see the result right in front of you.

I helped when deciding about the frontend. The placement and the style. I implemented in html the start page which is where the information about the task is about.

MING

I learned about the MING stack in the project. It is more in the main report. The MQTT is the protocol where a message is published, it is then sent to a broker who forwards the message to the subscribed topics. Node-RED is a visual tool where nodes are connected with wires. It shows how the data flows. Which node that takes it in and which node that takes it out. In the picture it takes 3 nodes, which is a trigger to a function, shown in the second picture, then to a database. This database would be InfluxDB. The function takes in 3 data points. The screenshots are from Docker when I learned about MING. This was to observe. Which made it easier for me when implementing the simulation.



Properties

Name

function 1

Setup

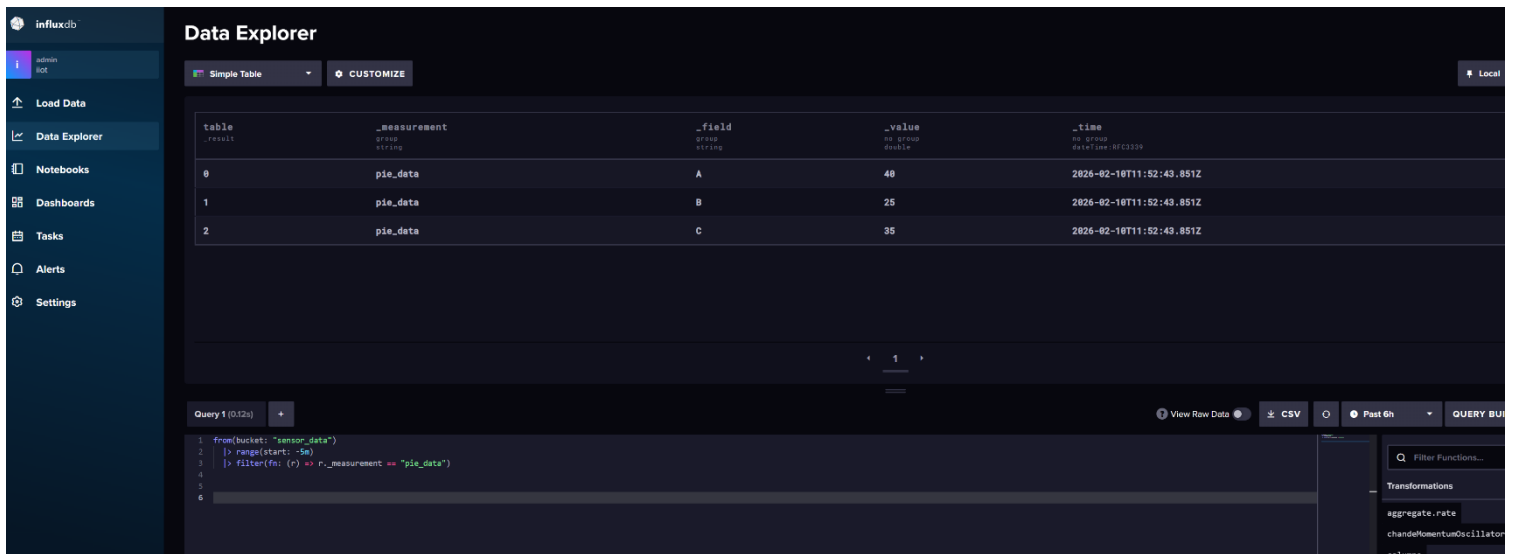
On Start

On Message

On Stop

```
1 msg.measurement = "pie_data";
2
3 msg.payload = {
4   A: 40,
5   B: 25,
6   C: 35
7 };
8
9 return msg;
10
11
```

The picture below is how the InfluxDB page looks like, it saves timestamped data. It works like a database. It shows the time when the data arrived.

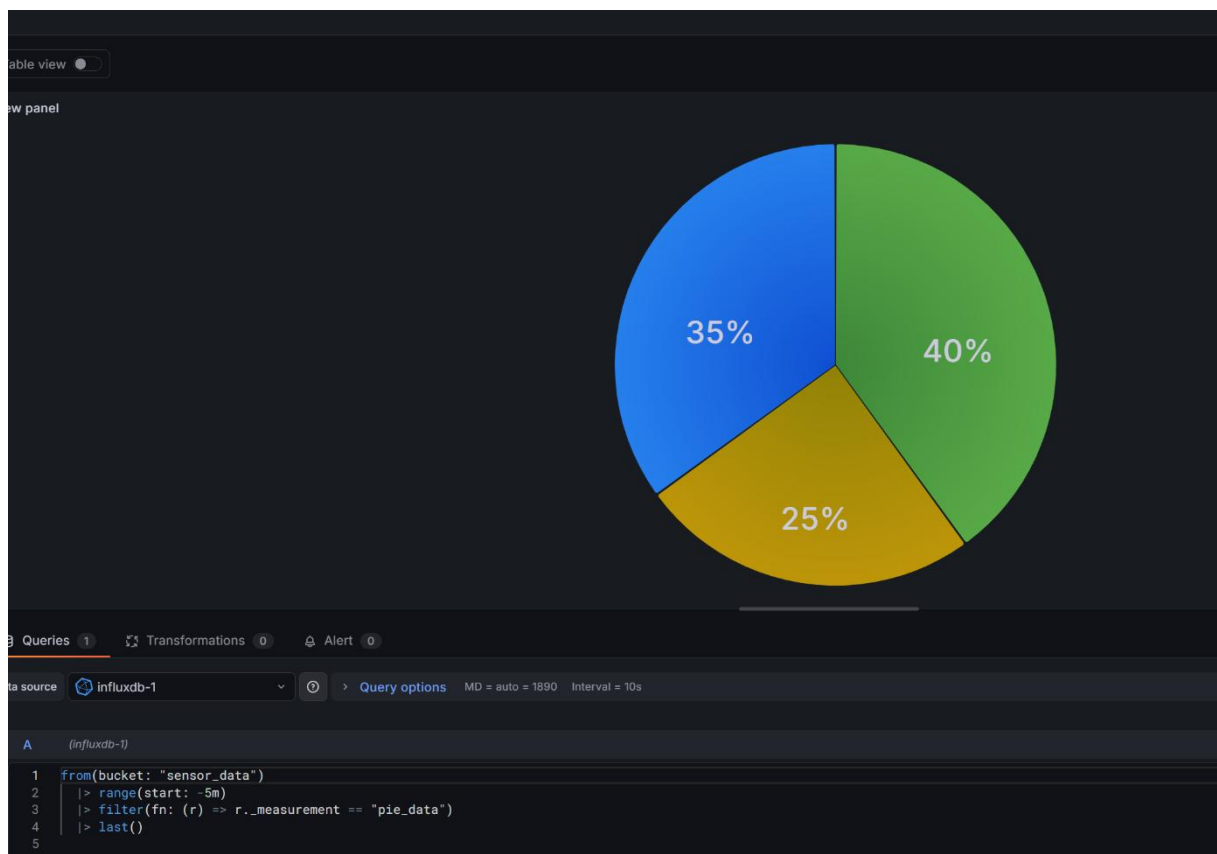


The screenshot shows the InfluxDB Data Explorer interface. On the left is a sidebar with navigation options: Load Data, Data Explorer (selected), Notebooks, Dashboards, Tasks, Alerts, and Settings. The main area is titled 'Data Explorer' and has a 'Simple Table' view selected. It displays a table with the following data:

table	_measurement	_field	_value	_time
sensor_data	pie_data	A	40	2026-02-10T11:52:43.851Z
sensor_data	pie_data	B	25	2026-02-10T11:52:43.851Z
sensor_data	pie_data	C	35	2026-02-10T11:52:43.851Z

Below the table, there is a query editor showing a query for 'sensor_data' filtered by 'pie_data'. The query is: `from(bucket: "sensor_data") |> range(start: -5m) |> filter(fn: (r) => r._measurement == "pie_data")`. The interface also includes options to view raw data, export to CSV, and a 'Past 6h' time range selector.

Grafana can be read in the main report, but the result would be represented in different ways such as this below.



Source list

- [1] WalkingTree, «Performance Comparison: Blazor vs. JavaScript Frontend Technologies,» WalkingTree, 28 July 2026. [Internett]. Available: <https://walkingtree.tech/performance-comparison-blazor-vs-javascript-frontend-technologies/>. [Found 4 April 2026].
- [2] ELPROCUS, «Difference between ESP32 s3 vs ESP32,» ELPROCUS, [Internett]. Available: <https://www.elprocus.com/difference-between-esp32-s3-vs-esp32/>. [Found 3 Mars 2026].
- [3] Mahler, «IoT Made Easy with Node-RED and InfluxDB,» Influxdata, 2 July 2025. [Internett]. Available: <https://www.influxdata.com/blog/iot-easy-node-red-influxdb/>. [Found 4 May 2026].
- [4] N. Griffin, «Hvorfor er MQTT en viktig del av IoT?,» Fiberroad, 14 October 2022. [Internett]. Available: <https://fiberroad.com/no/why-is-mqtt-an-important-part-of-iot/>. [Found 27 Mars 2026].